# ML System Design and Projects

Career preparation notes Sources: standard material, listed under References

### What this file is really about:

$$\text{clarify the goal} \rightarrow \text{data} \rightarrow \text{baseline} \rightarrow \text{model} \rightarrow \text{serving} \rightarrow \text{monitoring} \rightarrow \text{iterate}$$

**How to use this file.** Part A gives a framework, and worked case studies. Part B gives code for a production-style skeleton. Part C lists projects.

### Step-up path

**You need.** Evaluation, deployment and monitoring basics.

1. Clarify the goal and the metric.
2. Plan data and features.
3. Design retrieval and ranking.
4. Serve and test safely.
5. Estimate cost, and monitor.

**You can stop when** you can lead a complete ML system design discussion.

---

## Contents

# Part A Maths From the Ground Up

## A1 A framework for any design question

1. **Clarify the goal.** What business metric matters, what are the constraints, and how many users are there?
2. **Frame it as ML.** Classification, ranking, regression? What is a label, and when do we learn it?
3. **Data.** Sources, volume, freshness and bias.
4. **Features.** Which are available at prediction time?
5. **Baseline.** A simple rule first.
6. **Model.** Start simple and justify any complexity.
7. **Offline evaluation.** A metric tied to the goal, with a split matched to how the model is used.
8. **Serving.** Batch or online, latency and cost.
9. **Online evaluation.** An A/B test with guardrail metrics.
0. **Monitoring.** Data drift, performance, and a rollback plan.

Say your assumptions aloud, and discuss trade-offs. Interviewers grade the reasoning.

---

## A2 Case study: a recommendation feed

Goal: maximise long-term engagement, with limits on content quality. Use two stages. **Retrieval** narrows millions of items to hundreds cheaply, with two-tower embeddings and approximate nearest neighbours. **Ranking** then scores these with a richer model. Offline: recall at $k$ for retrieval, and NDCG for ranking. Beware feedback loops, since the model only sees outcomes for what it showed. Reserve a little traffic for exploration.

---

## A3 Case study: fraud detection

Fraud is rare and adversaries adapt. Labels arrive weeks late. Use gradient-boosted trees on aggregate features, such as the count of transactions in the last hour. Evaluate with precision at the review capacity, and split by time. Optimise the threshold for cost, $c_{\text{fp}} \cdot \text{FP} + c_{\text{fn}} \cdot \text{FN}$. Retrain often, and monitor drift.

## A4 Case study: a document assistant

Use a retrieval pipeline over the documents, with access control enforced at retrieval. Evaluate retrieval and generation separately. Require citations, and treat retrieved text as data, because it can contain hidden instructions. Log questions for continuing evaluation.

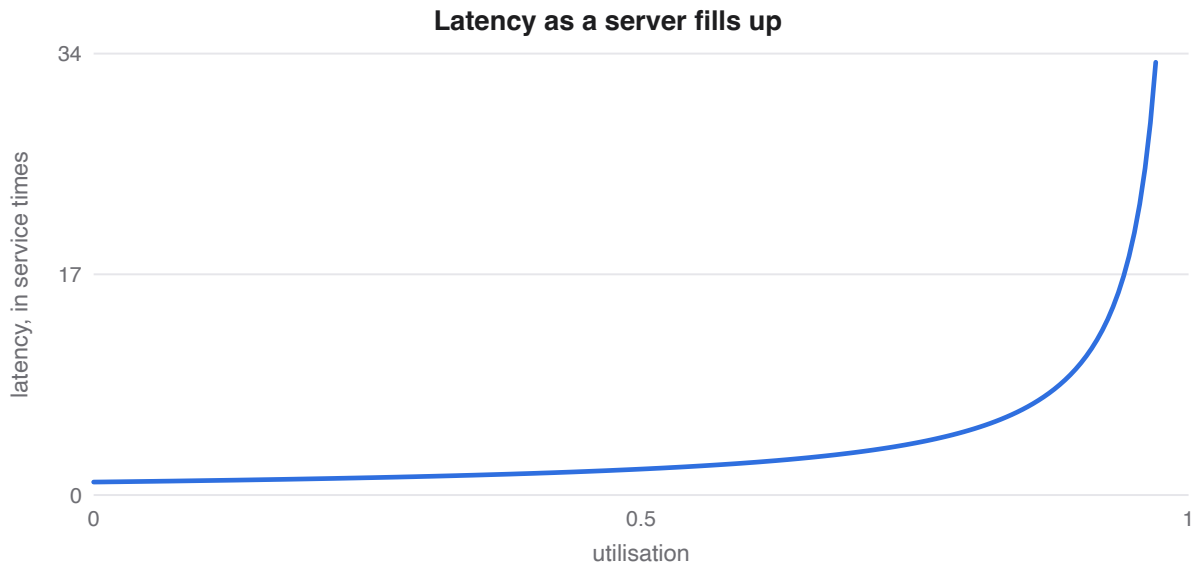
## A5 Case study: search ranking

Collect clicks as implicit feedback, and correct for position bias, since higher results get more clicks anyway. Use learning-to-rank with pairwise or listwise losses. Evaluate with NDCG:

$$\mathrm{DCG} = \sum_{i} \frac{2^{r_i} - 1}{\log_2(i+1)}.$$

## A6 Batch or online

Batch prediction is simple, and suits stable inputs. Online prediction handles fresh features, but needs low latency and a feature store. A **feature store** computes features once, and serves the same definitions to training and production, which prevents training-serving skew.



## A7 Deployment patterns

- **Shadow.** The new model runs but its output is not used.
- **Canary.** A small share of traffic gets the new model.
- **A/B test.** Random assignment measures the effect.
- **Rollback.** Keep the previous model ready.

## A8 Cost and scale

Estimate requests per second, latency, and the compute needed. For a model taking 20 ms on one CPU core, one core handles 50 requests per second at most, so 5000 requests per second needs at least 100 cores, and more for headroom.

## Part B In Code

### B1 A reproducible training pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import average_precision_score
from sklearn.model_selection import TimeSeriesSplit
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


def build(num, cat):
    pre = ColumnTransformer([
        ('num', make_pipeline(SimpleImputer(strategy='median'), StandardScaler()), num),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat),
    ])
    return make_pipeline(pre, HistGradientBoostingClassifier(random_state=0))


def evaluate(model, X, y, splits=5):
    scores = []
    for tr, te in TimeSeriesSplit(splits).split(X):
        model.fit(X.iloc[tr], y.iloc[tr])
        scores.append(average_precision_score(y.iloc[te], model.predict_proba(X.iloc[te])[:
    return sum(scores) / len(scores)
```

## B2 Serving a model

```
import joblib
from fastapi import FastAPI
from pydantic import BaseModel
import pandas as pd

app = FastAPI()
model = joblib.load('model.joblib')


class Row(BaseModel):
    amount: float
    country: str


@app.post('/predict')
def predict(row: Row):
    p = float(model.predict_proba(pd.DataFrame([row.model_dump()]))[0, 1])
    return {'probability': p, 'model_version': '1.0.0'}
```

## B3 A drift check

```
import numpy as np

def psi(reference, live, bins=10):
    edges = np.quantile(reference, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf
    p = np.histogram(live, edges)[0] / len(live) + 1e-6
    q = np.histogram(reference, edges)[0] / len(reference) + 1e-6
    return float(((p - q) * np.log(p / q)).sum())
```

## Part C Projects to Build

Each project should have a public repository, a clear README with the problem, the data, a baseline, the results, and what failed.

**P1. Tabular risk model.** Clean a real dataset, compare against a baseline, calibrate, and explain the predictions.

**P2. Image classifier with transfer learning.** Show error analysis, and a Grad-CAM view of the failures.

**P3. Retrieval assistant.** Answer from your own documents, with citations and an evaluation set.

**P4. GPT from scratch.** Train on a small corpus, and write up what each part does.

**P5. Deployed service.** Serve any model behind an API, with tests, a container, logging and a drift check.

**P6. Practice.** A request takes 10 ms of one core. How many cores handle 2000 requests per second, at most 50% utilisation?

> Each core handles 100 per second, or 50 at half utilisation, so $2000/50 = 40$ cores.